PRODUCT REQUIREMENTS DOCUMENT

Unified Grievance Intelligence & Resolution Platform

AI-Powered Civic Complaint Management for Indian Municipalities

Team	Code Yodha
Team Leader	Alina Khan
Members	Tanay Patil, Awwab Mhiaskar
Hackathon	NIRMAN — Amity University Mumbai
Domain	Web Development / GovTech
Problem Statement	Unified Grievance Intelligence & Resolution Platform
Version	1.0 — Production-Grade PRD
Date	11 April 2026
Status	FINAL — Submitted for Hackathon Review

Scope: This PRD covers all 12 functional pages of the platform — from citizen complaint ingestion to trust-based audit trails, silent crisis detection, and governance controls. All architecture references the locked tech stack defined in the system overview.

PAGE 1: Citizen Complaint Submission — All Channels

Purpose: Define the end-to-end intake pipeline for all seven complaint ingestion channels — Web, WhatsApp, Email, Twitter/X, Reddit, Telegram, and Voice — into a single unified complaints table in Supabase.

1.1 User Roles & Entry Points

Citizen (Web)	Authenticated or anonymous. Files complaint via React web app.
Citizen (Voice)	Uses browser-native Web Speech API on the React form for voice-to-text.
WhatsApp User	Sends message to registered Meta WhatsApp Business number. No app needed.
Email Filer	Sends email to a designated municipal inbox. Subject = complaint title.
Twitter/X User	Posts tweet with civic keywords or hashtags. Scraped automatically.
Reddit User	Posts to monitored subreddits (r/mumbai, r/delhi). Scraped automatically.
Telegram User	Messages the civic Telegram Bot. Bot relays to backend.

1.2 Channel-by-Channel Interaction Flows

1.2.1 Web App (React + Tailwind CSS)

The web complaint form is built in React with Tailwind CSS utility classes. The form renders in the citizen's selected language (see Section 1.5). Fields include:

- Complaint Type — dropdown pre-populated with categories: Water, Electricity, Roads, Garbage, Parks, Public Safety, Other.
- Description — free-text textarea with a minimum 15-word validation enforced on the frontend.
- Location — auto-detected via the browser Geolocation API (navigator.geolocation). A Leaflet/OpenStreetMap widget displays the detected pin. The citizen can drag the pin to correct it. Latitude and longitude are captured.
- Photo Attachment (optional) — image upload via <input type='file'> restricted to JPEG/PNG under 5 MB. Preview shown in-UI before submit.
- Anonymous Mode toggle — renders a visible UI checkbox. If toggled, the form does not send user_id to the backend. A hashed device fingerprint is computed client-side for rate limiting only and is never persisted.
- Language selector — dropdown supporting English, Hindi, Marathi, Tamil, Bengali, Gujarati, Kannada. Selection persists via localStorage.

- Voice input button — activates the Web Speech API. Transcribed text populates the description field in real time.

On submit, the React form calls POST /api/complaints/submit via Axios. A loading spinner is shown. On success, the citizen receives a unique tracking ID displayed on screen and sent to their registered email/phone if available.

1.2.2 WhatsApp (Meta Cloud API)

- A citizen sends a free-text message or voice note to the registered WhatsApp Business number.
- Meta's Cloud API fires a webhook POST to the Node.js backend endpoint POST /api/ingestion/whatsapp.
- The backend parses the JSON payload: message body, sender phone number (hashed as user identifier), and timestamp.
- If the message is a voice note, the media URL is fetched, transcribed via Gemini Speech-to-Text, and the transcript becomes the complaint description.
- Location is extracted if the citizen shares a WhatsApp location pin (latitude, longitude from the payload).
- A confirmation WhatsApp message is sent back to the citizen with their tracking ID via the same Meta Cloud API.

1.2.3 Email (SMTP / Nodemailer Inbound)

- Citizens email a designated municipal address (e.g., complaints@platform.gov.in).
- SendGrid Inbound Parse (or equivalent SMTP listener) fires a webhook POST to POST /api/ingestion/email with a parsed JSON body.
- Subject line is mapped to complaint_title. Email body is mapped to description. Any image attachment is stored to Supabase Storage and its public URL linked to the complaint.
- Sender email is used as the citizen identifier. If no prior account exists, a soft account is created with role = 'email_citizen'.
- A reply email is sent via Nodemailer confirming receipt and providing the tracking ID.

1.2.4 Twitter/X (v2 Filtered Stream + Scheduled Search)

- The backend runs a persistent filtered stream subscription on the Twitter v2 API, filtering by keywords: water outage, pothole, garbage, power cut, nala, along with civic hashtags such as #BMCComplaints and #MyGovComplaints.
- Each matching tweet is forwarded as a synthetic complaint payload to the internal ingestion pipeline with source = 'twitter_stream'.
- A separate node-cron job runs every 15 minutes calling GET /2/tweets/search/recent with broader keyword queries. Results are deduplicated against tweets already ingested in the current 15-minute window.
- Location is extracted from the tweet's geo field if available. If not, the extracted_location field is populated by Gemini's NLP output (see Section 1.7).
- No reply is sent to the Twitter user. The tweet is ingested silently.

1.2.5 Reddit API (snoowrap)

- A node-cron job runs every 30 minutes querying monitored subreddits (r/mumbai, r/delhi, r/bangalore, r/hyderabad, r/pune) for new posts containing civic keywords.
- Posts are fetched via the snoowrap Reddit API wrapper. Each qualifying post's title + body is extracted as complaint text. The subreddit serves as a loose location proxy.
- Ingested with source = 'reddit_auto'. No reply or action is taken on the Reddit post.

1.2.6 Telegram Bot

- Citizens message the platform's Telegram Bot. The bot is implemented via the Telegram Bot API and runs as a long-polling Node.js service.
- Incoming messages are forwarded to POST /api/ingestion/telegram. The Telegram user ID is hashed and used as the citizen identifier.
- The bot replies with the tracking ID upon successful ingestion.
- Voice messages received via Telegram are transcribed using Gemini Speech-to-Text before ingestion.

1.3 Multilingual Handling (Gemini + Web Speech API)

- The React frontend uses the Web Speech API for voice input. The recognition language is set based on the citizen's selected language preference.
- All non-English text — whether typed or transcribed — is passed to Gemini with the instruction: Detect the language of this text, then translate it to English. Return both the original language name and the English translation.
- The English translation is stored in complaints.description_en. The original text is stored in complaints.description_original. The detected language is stored in complaints.language.
- All AI processing (categorization, spam filtering, severity scoring) operates exclusively on description_en to ensure consistent model behavior.
- The citizen-facing UI renders status updates and notifications in the original language using Gemini translation at display time.

1.4 Anonymous / Whistleblower Mode

- The citizen toggles Anonymous Mode on the React form before submitting.
- Client-side: user_id is excluded from the POST body. A one-time anonymous_token (UUID v4) is generated client-side and included instead.
- Server-side: The backend sets user_id = NULL and is_anonymous = TRUE on the complaints row. No IP address is logged. No email or phone confirmation is sent.
- AES-256 encryption is applied to description_en before storage. The encryption key is stored separately in a restricted Supabase Vault, accessible only to a designated senior admin role.
- Rate limiting is enforced using a hashed device fingerprint (browser canvas hash + user agent hash). The fingerprint is computed server-side from request headers and is not stored anywhere. It exists only in the rate-limit Redis key for one hour, then expires.
- Officers and ward admins can view and act on anonymous complaints but cannot see or retrieve the identity of the filer. The is_anonymous flag is visible to them.

1.5 Backend Ingestion Pipeline (Node.js + Express)

All channels converge at a single internal function normalizeComplaint(payload) which produces a standardized object before any database write:

Normalized Complaint Object Fields

Field	Type	Source
source	ENUM	Web, WhatsApp, Email, Twitter, Reddit, Telegram
raw_text	TEXT	Original untranslated message
description_en	TEXT	Gemini-translated English version
description_original	TEXT	Text in original language
language	VARCHAR	Gemini-detected language
user_id	UUID / NULL	Authenticated user ID or NULL (anonymous)
anonymous_token	UUID / NULL	Only for anonymous mode
is_anonymous	BOOLEAN	TRUE if anonymous mode
lat	FLOAT8	GPS latitude (from GPS, WhatsApp pin, or tweet geo)
lng	FLOAT8	GPS longitude
media_url	TEXT / NULL	Supabase Storage URL for attached photo/voice
scraped_at	TIMESTAMPTZ	Timestamp of earliest public signal (social media)
created_at	TIMESTAMPTZ	Server timestamp of ingestion
status	ENUM	Default: pending_review
channel_metadata	JSONB	Raw channel-specific metadata (tweet ID, WhatsApp msgId, etc.)

After normalization, the pipeline proceeds in this exact sequence: rate limit check → language detection and translation → spam pre-check (regex only, no Gemini call yet) → database write to complaints table → async Gemini spam classification triggered as background job → confirmation dispatch to citizen.

1.6 Database Schema — complaints Table

complaints Table (Supabase PostgreSQL)

Column	Type	Notes
id	UUID PK	gen_random_uuid()
source	TEXT	Ingestion channel enum
raw_text	TEXT	Original message body
description_en	TEXT	English translation
description_original	TEXT	Original language text
language	VARCHAR(20)	Detected language code
user_id	UUID FK / NULL	FK → users.id. NULL for anonymous
is_anonymous	BOOLEAN	Default FALSE
anonymous_token	UUID / NULL	Whistleblower session token
lat	FLOAT8	Latitude
lng	FLOAT8	Longitude
geom	GEOMETRY(Point,4326)	PostGIS point for spatial queries
media_url	TEXT	Supabase Storage public URL
category	TEXT	Populated post-categorization
severity	TEXT	LOW / MEDIUM / HIGH
priority_score	INTEGER	1–5 composite score
spam_flag	BOOLEAN	TRUE if filtered as spam
spam_confidence	FLOAT4	Gemini spam confidence 0.0–1.0
status	TEXT	pending_review / active / merged / spam / resolved
master_ticket_id	UUID FK	FK → master_tickets.id
scraped_at	TIMESTAMPTZ	Earliest public signal timestamp

created_at	TIMESTAMPTZ	Server ingestion timestamp
channel_metadata	JSONB	Raw channel payload
sla_start	TIMESTAMPTZ	MIN(scraped_at, created_at)

1.7 AI Involvement

- Language Detection & Translation (Gemini): Called synchronously during normalization for non-English text. Produces description_en, language code.
- Speech-to-Text (Web Speech API / Gemini STT): Activated for voice input on web and for WhatsApp/Telegram voice notes. Gemini STT is used as the server-side fallback when Web Speech API is unavailable.
- Spam Pre-Classification (Gemini): Triggered asynchronously as a background job after the complaint is written to the database with status = pending_review. Result updates spam_flag and spam_confidence.
- Categorization & Severity (Gemini): Triggered as a second async job after spam classification passes. Result updates category, severity, and priority_score.

1.8 Validation, Rate Limiting & Abuse Prevention

Min Description Length	15 words enforced at frontend (React) and re-validated at backend.
File Type	Attachment restricted to JPEG / PNG only. File size limit: 5 MB.
Rate Limit (Authenticated)	5 complaints per user per hour. Enforced via Express rate-limiter middleware with Supabase-backed counters.
Rate Limit (Anonymous)	5 per hashed device fingerprint per hour. Fingerprint computed server-side, never stored.
Rate Limit (Social Media)	Social media scrapers are governed by respective API rate limits (Twitter: 500k tweets/month on Basic tier). No additional rate limiting applied.
Duplicate Submission (Naive)	Frontend disables submit button after first click. Backend rejects identical (user_id + description hash) submissions within 5 minutes.
CAPTCHA	Shown on anonymous web submissions after 3 complaints from the same fingerprint within an hour.

1.9 Failure Scenarios & Handling

Failure Handling Matrix

Failure Scenario	System Behaviour	User Impact

GPS location unavailable	Location field left as NULL. Complaint proceeds. Officer prompted to update location on review.	Citizen sees 'Location not detected' warning, can type manually.
Gemini STT transcription fails	Voice note stored as media_url. Backend sets transcription_failed = TRUE. Admin queue notified.	Citizen sees 'Voice processing pending' status.
WhatsApp webhook delivery failure	Meta Cloud API retries up to 3 times with exponential backoff. After 3 failures, the message is logged to a dead-letter queue.	Citizen receives no confirmation. Expected to retry.
Gemini API rate limit hit	Complaint status stays pending_review. A retry queue (Bull/BullMQ) retries after 60 seconds up to 5 times.	No user-visible impact. Processing delayed.
Supabase write failure	Express error handler returns HTTP 503. Client retries once automatically. Error logged to audit_log.	Citizen sees 'Submission failed, please retry' toast.
Oversized attachment	Backend rejects with HTTP 413 before Supabase Storage upload. Error returned to client.	Citizen sees 'File too large (max 5MB)' error.
Anonymous AES encryption failure	Complaint is NOT written. Error surfaced to citizen. Encryption is non-negotiable for anonymous mode.	Citizen sees 'Anonymous submission failed' and is prompted to try again.

1.10 Assumptions

1. All social media API access (Twitter v2, Reddit, Telegram) is configured and credentials are stored as environment variables in the Node.js backend.
2. WhatsApp Business API approval is obtained from Meta. A verified phone number is registered.
3. Supabase project is provisioned with PostGIS and pgvector extensions enabled.
4. For the hackathon, synthetic complaints are pre-loaded for demonstration. Real citizen accounts and ward data are mocked.
5. Gemini API key is available with sufficient quota for concurrent async jobs.
6. The browser Geolocation API is permitted by the citizen. If denied, manual location entry is required.
7. Voice input via Web Speech API is tested only on Chrome and Edge browsers. Firefox fallback uses Gemini STT exclusively.

PAGE 2: Spam Filtering & Validation Layer

Purpose: Every complaint entering the platform must pass a two-stage filter before it is made visible to officers or affects any SLA timer. This layer prevents noise, abuse, and bot submissions from polluting the complaint pipeline.

2.1 Rule-Based Filtering (Stage 1 — Zero API Cost)

Stage 1 executes synchronously at ingestion time, before any Gemini API call. It applies deterministic regex and heuristic rules:

- Minimum word count: Description must contain at least 15 words. Complaints with fewer than 15 words are auto-rejected with rejection_reason = 'too_short'.
- Repeated characters: Text containing 5+ consecutive identical characters (e.g., 'heeeeelp', 'aaaaaaa') is flagged.
- All-caps text: Text that is more than 80% uppercase characters is flagged as potential spam/bot.
- Promotional patterns: Regex matches common promotional phrases ('buy now', 'click here', 'free offer', 'limited time', phone numbers in non-complaint context) result in immediate reject.
- Known bot signatures: User agents or message patterns matching known bot fingerprints are rejected.
- URL-only messages: Complaints consisting only of URLs with no description text are rejected.
- Profanity threshold: High-density profanity (>40% of words) without a civic complaint context results in flagging for human review, not auto-rejection.

Outcome: Complaint either passes to Stage 2 (Gemini), is auto-rejected (spam_flag = TRUE, status = spam), or is flagged for human review (status = pending_human_review).

2.2 Gemini-Based Classification (Stage 2 — AI Safeguard)

Stage 2 is triggered asynchronously after the complaint is written to the database. The complaint is sent to Gemini with the following structured prompt:

Gemini Prompt Structure: You are a civic complaint validator for an Indian municipal grievance platform. Classify the following text as exactly one of: COMPLAINT or NOT_COMPLAINT. A COMPLAINT describes a real civic issue (infrastructure, utilities, safety, sanitation). NOT_COMPLAINT includes spam, promotional content, political messaging, personal grievances unrelated to civic services, or gibberish. Return ONLY a JSON object: { 'classification': 'COMPLAINT' or 'NOT_COMPLAINT', 'confidence': 0.0 to 1.0, 'reason': one sentence }. Text: [description_en]

Confidence ≥ 0.85	Marked as valid COMPLAINT. Proceeds to categorization pipeline.
Confidence 0.65–0.84	Classified but flagged: spam_confidence stored. Routes to human review queue if COMPLAINT; soft-rejected if NOT_COMPLAINT.

Confidence < 0.65	Auto-rejected. Stored with spam_flag = TRUE, status = spam. Not visible to officers.
Gemini returns NOT_COMPLAINT (any confidence)	Soft-rejected. Stored with status = spam. Citizen can appeal via web form.
Gemini API error / timeout	Complaint remains in pending_review. Retry queued. Never auto-rejected due to API failure.

2.3 Confidence Thresholds

Spam Classification Threshold Matrix

Threshold Range	Classification	Action	Officer Visibility
≥ 0.85 COMPLAINT	Valid	Proceeds to categorization	Yes
0.65–0.84 COMPLAINT	Uncertain — Valid	Human review queue	No (pending)
0.65–0.84 NOT_COMPLAINT	Uncertain — Spam	Soft reject, citizen can appeal	No
< 0.65 (any)	High-confidence spam	Auto-reject, no appeal	No
API error	Unresolved	Retry queue, stays pending	No

2.4 Voice Complaint Validation

- Voice complaints (from web or WhatsApp/Telegram voice notes) are transcribed first. The resulting transcript is treated identically to typed text and passes through both Stage 1 and Stage 2 filters.
- If the transcription confidence returned by Gemini STT is below 0.70, the complaint is flagged with transcription_low_confidence = TRUE and routed to human review — it is not auto-rejected.
- Voice complaints with a duration under 3 seconds are rejected as likely accidental activations.
- Audio files are stored at Supabase Storage regardless of transcription outcome, so a human reviewer can listen to the original.

2.5 Crowd Validation Integration

Crowd validation is a tertiary layer applied to complaints that survive spam filtering and have a valid GPS location. It does not gate the complaint from officer visibility — it adjusts priority.

- Upon ingestion, PostGIS identifies all registered citizens within 500 metres of the complaint's coordinates.

- Up to 10 nearby citizens receive a Firebase FCM push notification: 'A [category] complaint was filed near you. Have you seen this issue? Confirm or Dismiss.'
- Confirmations increase the complaint's priority_score by up to 1 point (capped at 5). Dismissals decrease it by up to 0.5.
- Citizens with a history of accurate vote outcomes (tracked in users.validation_accuracy) receive a vote weight of 2.0 versus the default weight of 1.0.
- Crowd votes are stored in complaint_votes(complaint_id, user_id, vote ENUM['confirm','dismiss'], weight, created_at).
- A minimum of 3 confirmations is required for the priority_score to be incremented. Single isolated votes have no effect.

2.6 Human Review Triggers

The following conditions route a complaint to the human review queue (status = pending_human_review), visible in the Admin Dashboard's Review Panel:

- Gemini confidence 0.65–0.84 for either COMPLAINT or NOT_COMPLAINT.
- Transcription low confidence flag.
- Profanity density above threshold but below auto-reject level.
- Anonymous complaint with no GPS location AND no photo (high-risk for misuse).
- Complaint text is entirely in a language not in the supported list (may be a foreign-language tourist complaint — valid, but needs manual routing).
- Complaint contains sensitive keywords (bomb, threat, explosion) — routed to both human review and Public Safety category fast-track.

2.7 Database Flags

Spam-Related Database Fields on complaints Table

Column	Type	Values
spam_flag	BOOLEAN	TRUE if classified as spam at any stage
spam_confidence	FLOAT4	Gemini confidence score 0.0–1.0
spam_stage	TEXT	rule_based or gemini
spam_reason	TEXT	One-sentence Gemini reason or regex rule name
status	TEXT ENUM	pending_review, pending_human_review, active, spam, merged, resolved
transcription_low_confidence	BOOLEAN	TRUE if STT confidence < 0.70

appeal_filed	BOOLEAN	TRUE if citizen has filed an appeal against spam rejection

2.8 Fallback Logic

- If Gemini Stage 2 is unavailable (API key expired, quota exhausted, network error), complaints are NOT auto-rejected. They remain in pending_review and are processed in bulk once the API recovers.
- An admin alert is fired via Nodemailer when more than 50 complaints accumulate in pending_review status due to Gemini unavailability.
- Stage 1 rule-based filtering always runs regardless of Gemini availability. Only obvious spam is rejected in this degraded mode.
- For social media scraped complaints, Gemini unavailability causes the entire scraper batch to be held in a staging table (complaints_staging) and not written to the main complaints table until validation is complete.

PAGE 3: Categorization, Severity & Priority Engine

Purpose: After spam validation, every active complaint is automatically classified into a department category, assigned a severity level, and given a numeric priority score. This determines which officer receives the complaint and how urgently they must act.

3.1 Gemini Classification Inputs

The categorization job receives the following fields as context for the Gemini API call:

- description_en — the English-translated complaint text.
- extracted_location — location name extracted from the complaint text or reverse-geocoded from GPS coordinates.
- media_url — if a photo is attached, Gemini Vision is invoked separately to cross-verify the category (see Section 3.7).
- source — ingestion channel (used to weight confidence; official web filings weighted higher than auto-scraped social media).

Gemini Categorization Prompt: You are a civic complaint classifier for Indian municipal services. Classify the complaint into EXACTLY ONE category from: WATER, ELECTRICITY, ROADS, GARBAGE, PARKS, PUBLIC_SAFETY, HEALTH, DRAINAGE, TRANSPORT, OTHER. Also extract: 1) affected_location_name (string, the specific location or address mentioned), 2) severity (CRITICAL / HIGH / MEDIUM / LOW), 3) emergency_keywords (array of any emergency words found: fire, flood, accident, collapse, gas leak, body, death, injury). Return ONLY valid JSON. Complaint: [description_en] Location context: [extracted_location]

3.2 Priority Scoring Formula

Priority Score Formula: priority_score = ROUND((keyword_score × 0.30) + (sentiment_score × 0.20) + (cluster_size_score × 0.30) + (location_impact_score × 0.20)) — Result is clamped to integer range 1–5.

Priority Score Component Definitions

Component	Weight	Source	Scale
keyword_score	30%	Gemini emergency_keywords array. Each emergency keyword adds +0.5 to raw score.	0.0 – 1.0
sentiment_score	20%	Gemini sentiment analysis of description_en. Strongly negative =	0.0 – 1.0

		1.0, neutral = 0.5, positive = 0.0.	
cluster_size_score	30%	Normalized count of complaints in the same master_ticket. Formula: MIN(cluster_size / 50, 1.0). 50+ complaints = max score.	0.0 – 1.0
location_impact_score	20%	PostGIS check: is the complaint within 200m of a hospital, school, or government facility? Yes = 1.0. Else population density percentile of the ward / 100.	0.0 – 1.0

Priority Score to Action Mapping

Score	Label	SLA Tier	Officer Alert
5	CRITICAL	Tier 1 — Immediate (2–4 hours)	Instant SMS + push to department head
4	HIGH	Tier 2 — Same Day (8 hours)	Push notification to assigned officer
3	MEDIUM	Tier 3 — 48 hours	Email notification to assigned officer
2	LOW	Tier 4 — 7 days	Dashboard update only
1	MINIMAL	Tier 5 — 30 days	Added to backlog, no active notification

3.3 Emergency Override Logic

If Gemini returns any item in emergency_keywords (fire, flood, accident, collapse, gas leak, body, death, injury, explosion), the following override applies regardless of computed priority_score:

- priority_score is force-set to 5 (CRITICAL).
- category is force-set to PUBLIC_SAFETY unless a more specific emergency category applies (e.g., WATER for flood, HEALTH for injury).
- An immediate escalation event is written to audit_log with action = 'emergency_override'.
- The complaint bypasses the normal assignment queue and is directly assigned to the department head of the relevant department.
- A Twilio SMS is fired to the on-call officer within 60 seconds of ingestion.

- The master ticket created from this complaint is tagged is_emergency = TRUE.

3.4 AI Misclassification Correction

- Officers can flag a categorization as incorrect via a 'Flag Category' button on the Officer Dashboard.
- Flagging opens a correction modal where the officer selects the correct category. The correction is written to category_corrections(complaint_id, original_category, corrected_category, officer_id, corrected_at).
- If 5 or more corrections accumulate for the same original_category → corrected_category pattern, an automated alert is sent to the platform admin noting a potential systematic Gemini misclassification.
- Gemini prompt templates can be updated by the platform admin without a code deployment, using a prompt_templates table in Supabase. The categorization job reads the active prompt at runtime.

3.5 Admin Override

- Ward Admins and City Admins can manually override category, severity, and priority_score on any complaint or master ticket.
- All overrides are written to audit_log with action = 'admin_override', old_value, and new_value fields populated. Overrides cannot be undone — a correction is a new audit event on top of the old one.
- A mandatory reason field must be filled before an override is saved.

3.6 Explainability & Audit

- The Gemini response JSON (category, severity, emergency_keywords, confidence) is stored verbatim in complaints.ai_classification_raw (JSONB column).
- On the Officer and Admin dashboards, a 'Why this priority?' tooltip displays a plain-language explanation derived from the stored JSON: e.g., 'Priority 4: Contains keywords suggesting infrastructure damage. 12 similar complaints in the same area. Near a school.'
- Every AI classification event is written to audit_log with action = 'ai_classification', actor_id = 'system', and the full Gemini response in new_value.

PAGE 4: Deduplication & Complaint Clustering

Purpose: Prevent duplicate complaints about the same physical issue from creating separate tickets. 200 citizens reporting the same pothole should produce one high-priority master ticket, not 200 noise entries that dilute officer workload.

4.1 Gemini Embeddings + pgvector Similarity

When a new complaint clears spam filtering and is categorized, the deduplication service is invoked:

8. The complaint's description_en is sent to Gemini's text embedding model (text-embedding-004 or equivalent). The resulting 768-dimensional vector is stored in complaints.embedding (pgvector column).
9. A cosine similarity query is run against all active complaints filed within the past 7 days in the same category:

pgvector Query Logic: SELECT id, description_en, lat, lng, master_ticket_id, 1 - (embedding <=> $new_embedding) AS text_similarity FROM complaints WHERE created_at > NOW() - INTERVAL '7 days' AND category = $category AND status != 'spam' ORDER BY text_similarity DESC LIMIT 10;

10. For each of the top-10 results, a combined cluster score is computed: cluster_score = (text_similarity × 0.60) + (geo_proximity_score × 0.40). geo_proximity_score = MAX(0, 1 - (distance_in_metres / 1000)) where distance is computed via PostGIS ST_Distance.
11. If cluster_score > 0.80, the new complaint is merged into the existing master ticket of the top-scoring match.
12. If no match exceeds 0.80, a new master_ticket is created and the complaint is linked to it.

4.2 Geo-Clustering via PostGIS

- In addition to per-complaint vector matching, a background job runs every 30 minutes to detect spatial clusters of unmerged complaints using PostGIS ST_ClusterKMeans.
- Complaints within 300 metres of each other, filed within the past 24 hours, in the same category, are candidates for cluster merging.
- The centroid of the cluster becomes the master_ticket's lat/lng. The master ticket is created if none exists.
- This geo-clustering pass catches cases where complaint text is too varied for vector similarity to trigger (e.g., some say 'no water' and others say 'tap dry since morning' — different embeddings, same problem).

4.3 Master Ticket Logic

master_tickets Table Schema

Column	Type	Notes

id	UUID PK	Primary key
category	TEXT	Complaint category
cluster_size	INTEGER	Number of linked complaints
lat	FLOAT8	Centroid latitude
lng	FLOAT8	Centroid longitude
geom	GEOMETRY	PostGIS point for spatial queries
priority_score	INTEGER	Recomputed as cluster grows
status	TEXT ENUM	open / assigned / in_progress / resolved / escalated
is_emergency	BOOLEAN	TRUE if emergency override triggered
sla_deadline	TIMESTAMPTZ	Computed from MIN(sla_start) across all linked complaints
assigned_officer_id	UUID FK	FK → users.id
department_id	UUID FK	FK → departments.id
created_at	TIMESTAMPTZ	Time of first complaint in cluster
resolved_at	TIMESTAMPTZ	Resolution timestamp

complaint_cluster_map Table Schema

Column	Type	Notes
complaint_id	UUID FK	FK → complaints.id
master_ticket_id	UUID FK	FK → master_tickets.id
linked_at	TIMESTAMPTZ	When this complaint was merged
cluster_score	FLOAT4	Score that triggered the merge

When a new complaint is merged into an existing master ticket, the master ticket's priority_score is recomputed using the updated cluster_size_score (reflecting the larger cluster). The sla_deadline is rechecked — if the new complaint's sla_start is earlier than the current sla_deadline base, the sla_deadline is updated to the earlier time.

4.4 SLA & Priority Impact of Clustering

- A master ticket with 50+ linked complaints receives the maximum cluster_size_score of 1.0, contributing 0.30 to priority_score.
- The SLA clock (sla_start) is always anchored to the earliest complaint in the cluster — specifically the minimum of all scraped_at timestamps across linked complaints. This enforces the 'First Signal SLA' USP.
- When cluster_size crosses threshold values (10, 25, 50), an automatic priority escalation event is written to audit_log and the assigned officer receives a push notification.

4.5 False Merge Prevention

- The 0.80 cluster_score threshold is deliberately conservative. At this threshold, a text_similarity of 1.0 (identical text) still requires a geo_proximity_score of 0.33 (within 670 metres) to merge. A nearby complaint about a completely different topic will have a low text_similarity, keeping the overall cluster_score below 0.80.
- If an officer believes two complaints have been incorrectly merged, they can 'Unmerge' a complaint via the Officer Dashboard. The complaint is re-linked to a new master ticket, and the event is recorded in audit_log.
- Complaints from different categories are never merged regardless of text or geo similarity. Category is a hard partition boundary for clustering.
- Social media scraped complaints are given a merge_confidence_penalty of -0.05 to their cluster_score. They can merge with existing tickets but require slightly higher confidence to seed a new master ticket.

PAGE 5: Officer Assignment & SLA Tracking

Purpose: Once a master ticket is created, the system must assign it to the right officer in the right department, enforce SLA timers tied to the earliest public signal, and auto-escalate when deadlines are breached.

5.1 Ward-Level Assignment

- Each master ticket carries a centroid lat/lng. The backend uses a PostGIS ST_Within query to determine which ward boundary the ticket falls in.
- Ward boundaries are stored in the wards table as GEOMETRY(Polygon) columns populated from GIS shapefiles (BMC ward data or synthetic equivalents for the hackathon).
- The ticket is assigned to the department responsible for the complaint's category in that ward, via the department_ward_map table linking department_id, ward_id, and the category they handle.

5.2 Officer Capacity Model

- Each officer has a current_load counter (active tickets assigned) stored in users.active_ticket_count.
- Maximum concurrent load per officer is configurable per department (default: 20 active tickets).
- The routing.service.js selects the eligible officer in the matched department with the lowest active_ticket_count. If two officers have the same load, the one with the higher AATS sub-score (resolution rate) is preferred.
- On assignment, the officer's active_ticket_count is incremented. On resolution, it is decremented.

5.3 Government Officer vs Private Contractor Toggle

- The platform supports both government officers (role = 'officer') and approved private contractors (role = 'contractor') as ticket assignees.
- City Admins can toggle a department's allowed_assignee_types between 'government_only', 'contractor_only', or 'both'.
- Contractor profiles are pre-approved by city admins and stored in users table with role = 'contractor' and contractor_approval_status = 'approved'.
- Contractor performance (resolution rate, SLA compliance, AATS sub-score) is tracked identically to government officers and is publicly visible on the admin dashboard.
- Civic Memory (Page 12 / USP 8) tracks which contractor handled which tickets and surfaces patterns of repeat failures for the same contractor in the same ward.

5.4 No-Officer-Available Handling

- If all officers in the matched department have reached their maximum load, the ticket is placed in a department_overflow queue.

- The department head is notified via Nodemailer email and FCM push: 'Overflow queue has [N] unassigned tickets. Capacity adjustment required.'
- If the overflow queue exceeds 10 tickets, the City Admin is also notified.
- Emergency tickets (priority_score = 5) bypass the capacity check entirely and are assigned to the department head regardless of load.

5.5 SLA Timer Configuration & Escalation

SLA Configuration by Category

Category	Priority 5 (CRITICAL)	Priority 4 (HIGH)	Priority 3 (MEDIUM)	Priority 2–1 (LOW)
WATER	4 hours	8 hours	24 hours	72 hours
ELECTRICITY	2 hours	6 hours	24 hours	48 hours
ROADS	4 hours	24 hours	72 hours	7 days
GARBAGE	8 hours	24 hours	48 hours	7 days
DRAINAGE	4 hours	8 hours	48 hours	7 days
PUBLIC_SAFETY	1 hour	4 hours	12 hours	48 hours
PARKS	24 hours	72 hours	7 days	30 days
HEALTH	2 hours	8 hours	24 hours	72 hours
TRANSPORT	8 hours	24 hours	72 hours	14 days
OTHER	24 hours	72 hours	7 days	30 days

SLA timer starts at master_tickets.sla_start, which is set to the minimum scraped_at across all linked complaints. sla_deadline = sla_start + SLA_CONFIG[category][priority_score].

The slaCron.service.js runs every 15 minutes via node-cron and checks:

- Tickets where sla_deadline is within 2 hours: 'SLA Warning' notification sent to assigned officer via FCM push.
- Tickets where sla_deadline has passed and status is not resolved: status set to 'escalated'. Ticket reassigned to department head. Twilio SMS fired to officer and department head.
- Tickets where sla_deadline has passed by more than 48 hours: escalation_level set to 'city_admin'. City Admin dashboard shows a red badge. Nodemailer alert sent to City Admin.

5.6 Admin Control Over Assignment

- Ward Admins can manually reassign any ticket in their ward to a different officer or contractor.
- City Admins can reassign across departments and wards.

- All manual reassignments are written to audit_log. The previous officer's active_ticket_count is decremented; the new officer's is incremented.
- Admins can adjust SLA deadlines for individual tickets with a mandatory reason. The adjustment is logged as an audit event.

PAGE 6: Officer Dashboard & Proof of Resolution

Purpose: Define the Officer Dashboard's functional requirements — what officers see, what actions they take, and how resolution is verified through GPS-validated photo proof and Gemini Vision comparison.

6.1 Officer Dashboard — React + Tailwind Layout

The Officer Dashboard is a React SPA with Tailwind CSS. Role-based rendering via Supabase Auth (role = 'officer' | 'contractor'). Layout sections:

- Header bar: Officer name, department, ward, current active ticket count vs max load, logout.
- Ticket Queue panel: Sorted by priority_score DESC, then sla_deadline ASC. Each ticket card shows: complaint category, location name, cluster size (number of affected citizens), priority badge (colour-coded 1–5), SLA countdown timer (live, updates via Supabase Realtime), and status badge.
- Ticket Detail panel: Opens on clicking a ticket card. Shows full description, linked complaints list, photo attachments, citizen-reported location on a Leaflet map, AI classification reason tooltip, and action buttons.
- SLA Countdown: Displayed as HH:MM:SS live timer. Turns amber at 2 hours remaining. Turns red at 30 minutes remaining. Turns solid red with a pulsing animation on breach.
- Statistics sidebar: Officer's resolution rate this month, SLA compliance rate, average resolution time, and AATS sub-score.

6.2 Officer Actions

Available Officer Actions on a Ticket

Action	Trigger	Outcome	Audit Log Entry
Accept Ticket	Button click	status → in_progress. Officer confirms assignment.	action = 'ticket_accepted'
Add Progress Note	Text input	Note written to ticket_notes(ticket_id, officer_id, note, created_at).	action = 'progress_note_added'
Request Escalation	Button + reason form	Ticket flagged for admin review. Notification sent to ward admin.	action = 'escalation_requested'
Upload Proof	File input (see 6.3)	Before/after photos uploaded. GPS and Gemini Vision validation run.	action = 'proof_uploaded'

Mark Resolved	Button — only enabled after proof validation passes	status → resolved. resolved_at = NOW(). SLA compliance computed.	action = 'ticket_resolved'
Unmerge Complaint	Button on linked complaint	Complaint detached from master ticket. New master ticket created.	action = 'complaint_unmerged'
Flag AI Misclassification	Dropdown + correction	category_corrections record created.	action = 'category_corrected'

6.3 Proof Upload — Supabase Storage

- The 'Upload Proof' action requires exactly two images: a BEFORE photo (can be the original citizen-submitted photo if available) and an AFTER photo (uploaded by the officer after fix).
- Images are uploaded to Supabase Storage bucket proof-images, path: /{ticket_id}/before.jpg and /{ticket_id}/after.jpg.
- The React frontend uses Supabase JavaScript client's storage.from('proof-images').upload() method. A progress bar is displayed during upload.
- EXIF metadata is extracted server-side from the uploaded image: GPS coordinates (GPSLatitude, GPSLongitude) and timestamp (DateTimeOriginal).
- If GPS EXIF data is absent (e.g., camera without GPS), the officer is warned and prompted to manually confirm the location on a map pin. Manual confirmation is logged but lowers the officer's proof_quality_score.

6.4 GPS & Metadata Validation

- Backend validation checks that the EXIF GPS coordinates of the AFTER photo are within 500 metres of the master ticket's centroid coordinates (PostGIS ST_DWithin).
- If the distance exceeds 500 metres, the proof is rejected with reason = 'location_mismatch'. The officer is notified and prompted to upload a correct photo.
- The EXIF timestamp must be within the last 48 hours. Older timestamps indicate a pre-existing photo and are rejected with reason = 'stale_timestamp'.
- Three consecutive proof rejections on the same ticket trigger an automatic escalation to the ward admin.

6.5 Gemini Vision Comparison

After GPS and timestamp validation pass, the before and after images are sent to Gemini Vision with the following prompt:

> **Gemini Vision Prompt:** You are validating a civic complaint resolution. Image A is the 'before' state (problem reported). Image B is the 'after' state (officer's claimed fix). Does Image B show a meaningful, visible improvement relevant to the complaint category [category]? Consider:

absence of garbage, repaired road surface, restored infrastructure, etc. Return ONLY JSON: { 'verified': true or false, 'confidence': 0.0 to 1.0, 'reason': one sentence }.

verified = true, confidence ≥ 0.75	Proof accepted. Officer can mark ticket resolved.
verified = true, confidence 0.50–0.74	Proof tentatively accepted with low_confidence flag. Citizen is asked to rate resolution quality. If rating ≤ 2, ticket is re-opened.
verified = false (any confidence)	Proof rejected. Reason shown to officer. Officer must re-upload.
Gemini Vision API error	Proof stored, manual admin review required. Ticket stays in_progress.

6.6 Audit Logging for Officer Actions

Every officer action described in Section 6.2 writes a record to the audit_log table. This table is append-only (no UPDATE or DELETE allowed via Supabase RLS). The audit_log record structure:

audit_log Table Schema

Column	Type	Notes
id	UUID PK	gen_random_uuid()
ticket_id	UUID FK	FK → master_tickets.id
complaint_id	UUID FK / NULL	FK → complaints.id (if action is complaint-level)
actor_id	UUID FK / NULL	FK → users.id. NULL for system actions.
actor_role	TEXT	citizen, officer, contractor, admin, system
action	TEXT	Action name enum (e.g., ticket_accepted, proof_uploaded)
old_value	JSONB / NULL	State before action
new_value	JSONB / NULL	State after action
created_at	TIMESTAMPTZ	Server timestamp — immutable
ip_hash	TEXT / NULL	SHA-256 of actor IP. Never raw IP. NULL for anonymous actions.

PAGE 7: Citizen Tracking, Notifications & Feedback

Purpose: Citizens must be able to track their complaint in real time, receive proactive notifications at key lifecycle events, and provide feedback that affects department trust scores.

7.1 Complaint Status Lifecycle

Complaint Status State Machine

Status	Meaning	Next States	Citizen Visible
pending_review	Awaiting spam classification	active, spam, pending_human_review	Yes — 'Under review'
pending_human_review	Flagged, awaiting admin review	active, spam	Yes — 'Under review'
spam	Rejected as spam or noise	Terminal (appeal possible)	Yes — 'Not accepted'
active	Validated, in deduplication / assignment queue	assigned, merged	Yes — 'Received'
merged	Merged into a master ticket	in_progress (via master ticket)	Yes — 'Merged with similar complaints'
assigned	Master ticket assigned to officer	in_progress	Yes — 'Assigned to officer'
in_progress	Officer actively working on it	resolved, escalated	Yes — 'In progress'
escalated	SLA breached, escalated to dept head	in_progress, resolved	Yes — 'Escalated'
resolved	Proof verified, ticket closed	Terminal (re-open possible)	Yes — 'Resolved'

7.2 Supabase Realtime Updates

The Citizen Dashboard subscribes to live database changes using Supabase Realtime WebSocket connections. The React component subscribes on mount and unsubscribes on unmount:

Supabase Realtime Subscription Pattern: supabase.channel('complaint-[complaint_id]').on('postgres_changes', { event: 'UPDATE', schema: 'public', table: 'master_tickets', filter: 'id=eq.[master_ticket_id]' }, (payload) => updateStatusCard(payload.new)).subscribe()

Status card updates, SLA progress bar changes, and resolution proof displays all update in real time without page refresh. No polling is used — all updates are push-based via WebSocket.

7.3 Channel-Aware Notification Matrix

Notification Triggers by Channel

Lifecycle Event	Web Push (FCM)	Email (Nodemailer)	SMS (Twilio)	WhatsApp (Meta API)
Complaint received	Yes	Yes	No	Yes (if WhatsApp source)
Spam rejected	Yes	Yes	No	Yes (if WhatsApp source)
Merged with cluster	Yes	No	No	No
Assigned to officer	Yes	Yes	No	No
SLA breach warning (2hr)	Yes	No	No	No
Complaint escalated	Yes	Yes	Yes	No
Resolved — request feedback	Yes	Yes	Yes	Yes (if WhatsApp source)
Re-opened (proof rejected)	Yes	Yes	No	No

Notification content is rendered in the citizen's stored preferred language using a Gemini translation call at notification dispatch time. Templates are stored in notification_templates table and selected by event_type and language_code.

7.4 Citizen Feedback & Ratings

- When a ticket is resolved, the citizen receives a notification prompting them to rate the resolution on a 1–5 star scale.
- An optional comment field (max 200 characters) is available alongside the star rating.

- Ratings are stored in complaint_ratings(ticket_id, citizen_id, rating INT, comment TEXT, submitted_at).
- The rating window is open for 72 hours after resolution. After 72 hours, the prompt is dismissed and the ticket is permanently closed.
- If a citizen rates the resolution 1 or 2 stars, a 'Request Re-open' button appears. The citizen can explain why the issue is unresolved. A re-open request creates a new complaint linked to the original master ticket and restores status to 'in_progress'.

7.5 Trust Score Impact

Citizen feedback directly feeds the Aam Aadmi Trust Score (AATS) computed for each department. The formula and detailed mechanics are covered in Page 10. The citizen-facing impact is:

- If a citizen's re-open request is upheld (officer confirms the issue was unresolved), the department's SLA compliance score is penalised for that ticket.
- If a citizen's re-open request is dismissed by the admin as invalid, the citizen's vote_accuracy is decremented slightly, reducing their future crowd validation weight.
- Consistently high ratings (4–5) for a department cause their AATS score to rise, which is visible to all citizens on the department's public profile page.

PAGE 8: Admin Dashboard & Governance Controls

Purpose: Define three tiers of administrative control — Ward Admin, Department Admin, and City Admin — each with scoped data access, override capabilities, and governance tools that enforce accountability without enabling manipulation.

8.1 Admin Role Hierarchy

Admin Role Matrix

Role	Scope	Key Capabilities	Data Isolation
Ward Admin	Single municipal ward	View/assign tickets in their ward, approve human review queue, manage officer roster for their ward, view ward-level heatmaps.	Can only see complaints with centroid in their ward.
Department Admin	Single department city-wide	Monitor all tickets in their department across wards, manage officer/contractor roster for their department, view department AATS score.	Can only see complaints in their department category.
City Admin	All wards and departments	Full platform visibility, SLA breach oversight, approve new contractor profiles, manage prompt templates, view silent crisis panel, access immutable audit log, export reports.	Full access — no data isolation.

Role assignments are managed via Supabase Auth custom claims. JWT tokens include role and scope (ward_id or department_id) for server-side authorization in Express middleware.

8.2 Data Isolation

- Supabase Row Level Security (RLS) policies enforce data isolation at the database layer. Even if an API endpoint is mis-coded, the database will not return data outside an admin's scope.
- Ward Admin RLS: complaints.geom must be ST_Within(ward_boundaries WHERE ward_id = auth.ward_id).
- Department Admin RLS: complaints.category must match the department's assigned category list.
- City Admin RLS: No restriction — full table access.

- Audit log is read-only for all admin roles. No admin can delete or edit audit records.

8.3 Override Rules

- Ward Admins can override ticket category, priority, and officer assignment within their ward. All overrides require a written reason and are logged immutably.
- Department Admins can adjust SLA deadlines for tickets in their department. Reason required.
- City Admins can perform all overrides system-wide. Additionally, City Admins can bulk-dismiss spam-flagged complaints (e.g., during a known bot attack) via a bulk action with a batch audit log entry.
- No admin at any level can delete a complaint, delete an audit log entry, or alter a resolved_at timestamp. These operations are blocked at the database level.

8.4 SLA Breach Monitoring

- The Admin Dashboard's SLA Monitoring panel shows a real-time table of all escalated tickets, sorted by hours_overdue DESC.
- Colour coding: Yellow = 0–24 hours overdue, Orange = 24–72 hours overdue, Red = 72+ hours overdue.
- City Admins see a system-wide breach rate KPI card: percentage of tickets in the last 30 days that breached SLA, broken down by department.
- Ward Admins see the same metric scoped to their ward.
- A 'Force Escalate' button allows any admin to immediately reassign a breached ticket to the department head and send an urgent Twilio SMS.

8.5 Audit Log Access

- The Audit Log Viewer is available only to City Admins and is read-only. It supports filtering by ticket_id, actor_id, action type, and date range.
- Audit records are displayed in reverse chronological order with full old_value and new_value diffs.
- City Admins can export audit log records as CSV for RTI compliance or external review. The export is itself logged as an audit event.
- No audit record can be hidden, filtered out, or deleted from this view by any user including City Admins.

PAGE 9: Silent Crisis Detection & Heatmaps

Purpose: Detect underserved wards where low complaint volume does not mean no problems — it may mean no voice. Visualize all complaint data spatially on Mapbox heatmaps and surface silence as a distinct, actionable signal.

9.1 Silence Ratio Logic

The silence ratio is computed continuously in a Supabase materialized view that refreshes every 6 hours:

Silence Ratio Formula: silence_ratio = ward_population / NULLIF(active_complaint_count_last_30_days, 0). A ward with 600,000 people and 6 complaints has a silence_ratio of 100,000 — far higher than a ward with 80,000 people and 340 complaints (ratio = 235). Higher ratio = more likely underserved, not actually problem-free.

- ward_population is sourced from the ward_stats table, populated from Census 2011 data (or synthetic equivalents for the hackathon).
- active_complaint_count_last_30_days counts complaints with created_at > NOW() - INTERVAL '30 days' AND status != 'spam', grouped by ward via PostGIS ST_Within.
- A silence_threshold is configurable per city by the City Admin (default: 500). Wards with silence_ratio > threshold are flagged as silent_crisis = TRUE.

9.2 Population Normalization

- Raw complaint counts are never used without population normalization. A ward with 1000 complaints in a population of 1 million is less alarming than a ward with 10 complaints in a population of 50,000.
- The normalized complaint rate = (complaint_count / population) × 100,000 — expressed as complaints per 100,000 residents.
- This normalized rate is the primary metric shown on the Admin Dashboard's Ward Performance table.
- Ward-level population data is segmented by age group where available, to surface disproportionate impacts on elderly or child populations.

9.3 Mapbox Heatmaps (Frontend — React)

The heatmap is rendered on the Admin Dashboard using Mapbox GL JS within a React component. Three distinct visual layers:

Heatmap Layer Definitions

Layer Name	Colour	Data Source	Meaning

Active Complaint Density	Red → Amber → Green gradient	PostGIS ST_ClusterKMeans output on master_tickets	High volume of unresolved complaints in this area
Silent Crisis Zones	Grey pulsing overlay	ward_stats WHERE silent_crisis = TRUE	Low complaint volume relative to population
Resolved Cluster Layer	Green faded	master_tickets WHERE status = 'resolved'	Areas where complaints were resolved — shows coverage

The Mapbox layer source is a Supabase Edge Function endpoint returning GeoJSON from the master_tickets and ward_stats tables. The endpoint is called on map load and on 30-minute refresh intervals.

- Clicking any cluster on the heatmap opens a sidebar panel showing: complaint count, top categories, average resolution time, AATS score for relevant departments, and the silence ratio for that ward.
- A legend is always visible explaining the three visual layers and their meaning.

9.4 Admin Actions on Silent Crisis Zones

When a ward is flagged as a silent crisis zone, the City Admin Dashboard shows a Silent Crisis Action Panel with the following recommended actions:

- Deploy Field Survey Team: Admin can create a special field_survey ticket type (not citizen-filed) for officers to proactively inspect the ward and file observations.
- Enable IVR Intake: A toggle allows the ward to receive complaints via an IVR phone number (future integration — flagged as planned in hackathon scope).
- Partner NGO Outreach: Admin can tag a ward for NGO partnership outreach. The tag is stored in ward_stats.outreach_status.
- Lower Silence Threshold for Ward: Admin can set a ward-specific silence_threshold lower than the city default, making the ward appear in silent crisis alerts sooner.

9.5 Ethical Safeguards

- Silent crisis data is never used to deprioritize or reduce officer allocation to a ward. It is exclusively used to increase attention and resources.
- Ward-level silence data is not published on the public-facing citizen dashboard. It is internal to admin roles only, to prevent stigmatization of communities.
- Population data is used only in aggregate at ward level. No individual-level demographic data is used in any algorithm.
- City Admins can see the silence detection algorithm's parameters and the data sources it uses, ensuring the methodology is not a black box.

PAGE 10: Trust, Audit & Anti-Tampering Architecture

Purpose: Define the mechanisms that make data manipulation technically impossible — not just against policy. Cover the immutable audit trail, the Aam Aadmi Trust Score (AATS), public transparency features, and tamper prevention at the database level.

10.1 Immutable Audit Log

The audit_log table is the backbone of the platform's accountability guarantee. Its immutability is enforced at three independent layers:

13. Database layer (Supabase RLS): UPDATE and DELETE operations on audit_log are blocked for all roles including service_role via RLS policies. No application code can circumvent this.
14. Application layer: The Express backend has no DELETE /audit route and no UPDATE operation targeting audit_log anywhere in the codebase. Code review policy requires a security reviewer sign-off for any future change near audit_log.
15. Append-only write pattern: Every audit event uses an INSERT-only function. The function is defined as a Supabase Database Function with SECURITY DEFINER set to a restricted role that can only INSERT on audit_log.

Every platform action — complaint filing, assignment, SLA breach, proof upload, admin override, officer login, failed login attempt — writes a record. Nothing is excluded.

10.2 Aam Aadmi Trust Score (AATS)

AATS Formula: AATS = (resolution_rate × 0.40) + (sla_compliance_rate × 0.30) + (avg_citizen_rating_normalized × 0.30). Result is scaled to 0–100 and recomputed daily via a Supabase scheduled function.

AATS Component Definitions

Component	Weight	Definition	Data Source
resolution_rate	40%	Percentage of assigned tickets resolved within the last 90 days	master_tickets WHERE assigned_dept = dept AND status = resolved
sla_compliance_rate	30%	Percentage of resolved tickets that were resolved before sla_deadline	master_tickets WHERE resolved_at <= sla_deadline
avg_citizen_rating_normalized	30%	Average citizen star rating (1–5) normalized to 0.0–	complaint_ratings JOIN master_tickets

		1.0 scale. (avg_rating - 1) / 4	

- AATS is computed per department, not per officer. Officers contribute to their department's AATS score collectively.
- A department's AATS score is displayed publicly on the citizen-facing department directory page. No login required to view it.
- AATS history (daily snapshots) is stored in aats_history(department_id, score, computed_at) for trend tracking.
- A department whose AATS falls below 40 triggers a City Admin alert: 'Department X AATS critically low — review required.'

10.3 Public Transparency Features

- Every citizen who files a complaint can view a full, unfiltered timeline of their complaint's history on the tracking page — every status change, every assignment, every escalation — sourced directly from audit_log. No RTI request needed.
- The timeline is rendered read-only in the React Citizen Dashboard, with timestamps, actor roles (but not actor names — only roles like 'Ward Officer', 'City Admin'), and the nature of each action.
- Resolved complaints show the before/after proof photos publicly, unless the citizen flagged the complaint as sensitive (for anonymous/whistleblower complaints, no proof photos are publicly visible).
- Aggregate statistics (city-wide resolution rate, category breakdown, SLA compliance last 30 days) are displayed on a public Statistics page accessible without login.

10.4 Tamper Prevention Architecture

Tamper Prevention Mechanisms

Risk	Prevention Mechanism	Layer
Officer deletes complaint before audit	RLS blocks DELETE on complaints. Soft-delete only (status = archived) with audit entry.	Database (RLS)
Admin backdates resolution timestamp	resolved_at set by server-side trigger, never accepted from client payload. Client cannot send this field.	Backend (Express)
Officer changes SLA start time	sla_start computed server-side from MIN(scraped_at) and is never editable post-creation.	Backend + DB Trigger
Admin edits audit log entry	RLS blocks all UPDATE/DELETE on audit_log for every role.	Database (RLS)

Service account bypasses RLS	The Supabase service role is not used in application code. All operations use authenticated JWT with scoped roles.	Architecture
Proof photo GPS spoofed	EXIF extraction happens server-side. Photo is re-processed on upload, not trusted from client metadata.	Backend (Node.js)
Bulk spam-dismissal to hide complaints	Bulk dismiss action creates a batch audit entry. City Admin approval required for bulk actions > 10 tickets.	Admin controls + Audit

PAGE 11: Hackathon vs Production Scope

Purpose: Distinguish what is built and demonstrated at the hackathon from what would be required for real BMC / municipal deployment. This demonstrates architectural realism and judges' confidence in the team's awareness of production constraints.

11.1 Synthetic Data Usage

- All ward boundaries are synthetic GeoJSON polygons approximating Mumbai's 24 wards. In production, official BMC shapefile data would replace these.
- Population data per ward is derived from Census 2011 estimates. In production, this would be loaded from official municipal demographic data sources.
- Officer accounts and department structures are seeded with mock data. In production, this would require integration with the municipal HR system.
- Complaint data for the demo is pre-seeded with 500 synthetic complaints across 5 categories, distributed realistically across wards. Some wards are intentionally left with low complaint counts to demonstrate silent crisis detection.
- Twitter/X, Reddit, and Telegram complaint feeds are mocked using a Mock JS Service (socialScraper.service.js mock mode) that replays pre-recorded JSON fixtures on a timer.

11.2 Real BMC Integration Readiness

Integration Points for Production Deployment

Integration Point	Hackathon Approach	Production Requirement
Ward Boundary Data	Synthetic GeoJSON polygons	Official BMC GIS shapefiles via MCGM API or direct download
Officer / Staff Directory	Mock user accounts	LDAP / Active Directory integration with municipal HR system
WhatsApp Business Number	Test number on Meta sandbox	Verified WhatsApp Business API account with municipal branding
SMS (Twilio)	Twilio trial account	Production Twilio account with DLT-registered sender ID (TRAI compliance)
Email Ingestion	SendGrid sandbox	Dedicated municipal email address with DKIM/SPF for deliverability
Twitter/X API	Basic tier (limited)	Elevated or Academic access for higher tweet volume

Gemini API	Standard quota	Enterprise quota with SLA guarantees
Supabase	Free tier / Pro	Supabase Pro or dedicated PostgreSQL instance for data sovereignty
IVR / Phone Intake	Not implemented — planned	Twilio IVR flows or integration with IVRS system for voice-only citizens
RTI Data Export	CSV export via admin panel	CPIO-compliant automated RTI response system

11.3 Deployment Assumptions (Hackathon)

- Frontend (React + Tailwind) is deployed on Vercel. Build triggered via GitHub Actions on push to main branch.
- Backend (Node.js + Express) is deployed on Render. Environment variables for Supabase URL, Gemini API key, Twilio credentials, and Meta WhatsApp token are set in Render's environment configuration.
- Supabase project is on the Pro plan to support PostGIS, pgvector, Realtime, and Storage features simultaneously.
- No custom domain is configured for the hackathon demo. Vercel and Render default URLs are used.
- The cron jobs (SLA checker, social media scraper, silent crisis refresh) are run via node-cron within the Express process. In production, these would migrate to Supabase Edge Functions with pg_cron or a dedicated job scheduler.

PAGE 12: Metrics, Risks & Success Criteria

Purpose: Define the quantitative and qualitative measures by which the platform's success will be evaluated, the AI model performance targets, and the governance and technical risks with concrete mitigation strategies.

12.1 Platform KPIs

Key Performance Indicators

KPI	Target	Measurement Method	Frequency
Average complaint-to-resolution time	< 72 hours (Priority 3–4)	MEDIAN(resolved_at - sla_start) across resolved master_tickets	Daily
SLA compliance rate	≥ 95%	% of resolved tickets where resolved_at ≤ sla_deadline	Daily
Duplicate complaint reduction	< 5% unique complaints per cluster	cluster_size / total complaints ratio	Weekly
Citizen satisfaction (AATS sub-score)	≥ 3.8 / 5.0 average rating	AVG(complaint_ratings.rating) across all departments	Weekly
Spam filter false positive rate	< 2%	% of spam-flagged complaints successfully appealed	Weekly
Officer SLA breach rate	< 5% of assigned tickets	% of tickets escalated due to SLA breach / total tickets	Daily
Omnichannel ingestion success rate	≥ 99% for web; ≥ 95% for social/messaging	Successful writes to complaints table / total ingestion attempts	Real-time
Silent crisis ward detection rate	100% of wards with silence_ratio > threshold flagged	Ward coverage in ward_stats materialized view	6-hourly

12.2 AI Accuracy Targets

AI Model Performance Targets

AI Function	Accuracy Target	Measurement	Fallback if Target Missed

Spam Classification (Gemini)	≥ 92% precision	Human review of random 5% sample of auto-approved complaints	Lower confidence threshold → more human review
Auto-Categorization (Gemini)	≥ 90% agreement with officer corrections	category_corrections rate < 10%	Expand category correction feedback loop to retrain prompt
Deduplication (pgvector + geo)	< 3% false merge rate	Officer unmerge actions / total merges	Raise cluster_score threshold from 0.80 to 0.85
Severity Scoring (Gemini)	≥ 88% agreement with officer re-scores	Officer priority overrides rate	Review keyword_score and sentiment_score weights
Gemini Vision Proof Validation	≥ 85% agreement with human admin reviews	Admin manual override of vision verification / total verifications	Lower confidence threshold to 0.60 for tentative acceptance
Speech-to-Text (Web Speech / Gemini)	≥ 90% word accuracy for Hindi/English	Spot-check of 10% of transcribed voice complaints	Route low-confidence transcriptions to human review

12.3 SLA Compliance Targets

- Platform-level SLA target: 95% of all tickets resolved within their SLA deadline across all categories.
- Emergency ticket target: 100% of Priority 5 (CRITICAL) tickets responded to within SLA. Breach of this target triggers immediate City Admin alert.
- SLA compliance is broken down by ward and department in the Admin Dashboard. Underperforming wards/departments are flagged with a red badge on the city-wide SLA map.

12.4 Governance Risks

Governance Risk Register

Risk	Likelihood	Impact	Mitigation
Officers batch-resolving tickets without actual work	Medium	High	Gemini Vision proof validation required. GPS validation required. Citizen re-open within 72 hours re-opens ticket.
Admins bulk-dismissing complaints to improve AATS score	Low–Medium	High	Bulk dismiss requires City Admin approval. Every dismiss is audit-

			logged. Dismissals count against resolution_rate in AATS.
Gemini misclassification creating systemic routing errors	Low	Medium	Category correction feedback loop. Admin prompt template control. 5-correction alert threshold.
Anonymous complaint abuse (filing many false complaints)	Medium	Medium	Hashed fingerprint rate limiting. CAPTCHA after 3 anonymous submissions per hour. Stage 1 + Stage 2 spam filtering.
Social media scraper ingesting political content as civic complaints	Medium	Low	Spam Stage 2 (Gemini) catches non-civic content. Source tagged as social_media_auto — lower weight in priority scoring.
Data sovereignty concerns (Supabase is US-hosted by default)	High (for production)	High	Production deployment would use Supabase's India region or a self-hosted PostgreSQL instance to comply with data localisation requirements.
SLA timer gaming by delaying official acknowledgement	Low	High	SLA clock starts at earliest scraped_at (First Signal SLA USP). Official filing does not reset the clock.

12.5 Technical Risks & Mitigation

Technical Risk Register

Risk	Mitigation
Gemini API downtime	Bull/BullMQ retry queue. Complaints stay in pending_review, never auto-rejected. Admin alert after 50 queued.

pgvector similarity query latency at scale	HNSW index on embeddings column. Query scoped to 7-day window and same category. Tested up to 100,000 vectors.
Supabase Realtime connection limits	Connection pooling via Supabase Realtime. Citizens use a shared channel per master_ticket_id rather than per-complaint channels.
WhatsApp API webhook delivery failure	Meta retries 3x. After failure, dead-letter queue. Admin dashboard shows failed webhook count.
Node.js cron job drift on Render free tier (container sleeps)	Production: migrate cron jobs to Supabase pg_cron or a dedicated worker dyno. Hackathon: use Render paid plan to prevent sleep.
GPS EXIF spoofing via edited photos	Server-side EXIF re-extraction using ExifTool or sharp. Client-submitted metadata is ignored entirely.
Supabase Storage cost at scale	Photos compressed to max 1200px width before storage. Old resolved complaint photos archived after 1 year. Estimated storage: ~500 MB per 10,000 tickets.

END OF PRD — Code Yodha / NIRMAN Hackathon 2025

Unified Grievance Intelligence & Resolution Platform — Version 1.0